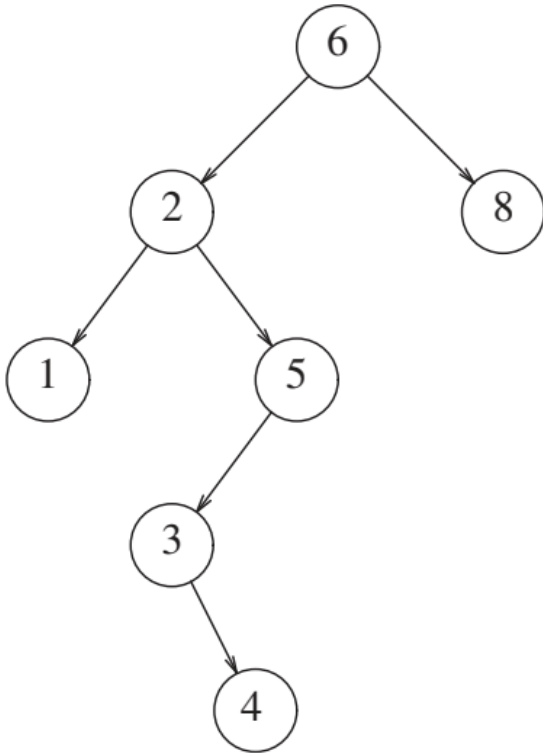


Example for Binary Search Tree: Remove (2), two children.

For example 5.3: if there is **the exist** BinarySearchTree's instance variable name **bst** contains the elements as the image following:

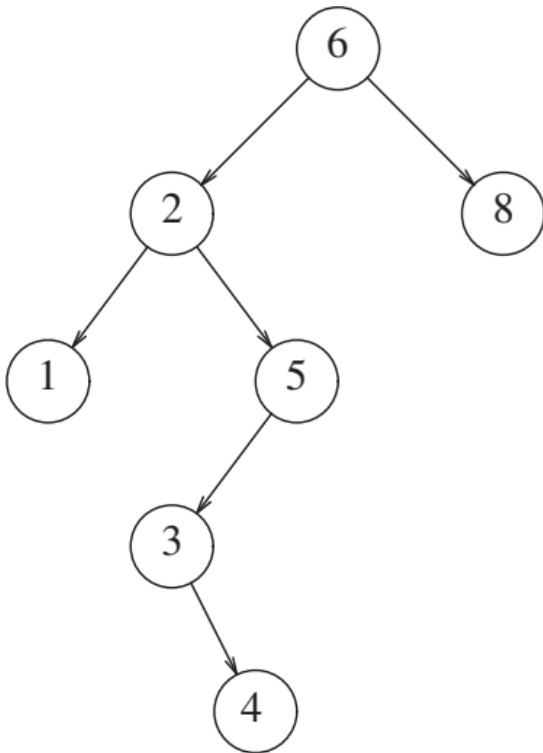


Write the steps for the `_delete_recursive()` function when remove 2 from the instance variable `bst`. In each round, consists of

- 1) what are the values of the parameters value and `current_node` of the `_delete_recursive()` function, and
- 2) what statements are executed under the if-else condition that is checked as true.

Each iteration must write 2 things.

Answer:



Each iteration for the `_delete_recursive()` function when remove 2

First round value = 2 , current_node = **6**
`current_node.left =`
`_delete_recursive(current_node.left, value)`

round 2: value = 2 , current_node = **2**
`temp_node = self._min_value_node(current_node.right)`

`current_node.value = temp_node.value` # นำค่า 3 มาทับตำแหน่ง 2

`current_node.right = self._delete_recursive(current_node.right, temp_node.value)`

[หลักการลบโหนดที่มีลูก 2 คน]:

* กฎ 3 ขั้นตอนลบโหนดที่มีลูก 2 ตัว (Two Children Removal):

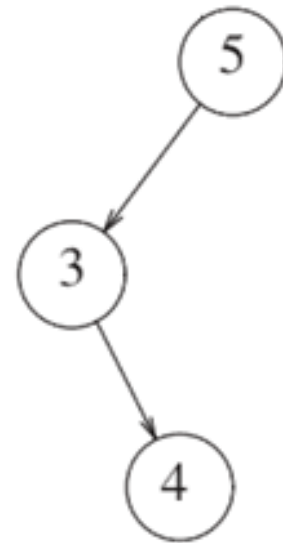
1. หา In-order Successor: `temp_node = self._min_value_node(current_node.right)`
2. คัดลอกค่ามาทับ: `current_node.value = temp_node.value` (นำ 3 มาแทน 2)
3. ลบโหนดตัวแทนเดิม: `current_node.right = self._delete_recursive(..., temp_node.value)`



Each iteration for the `_min_value_node()` function to find minimum of right subtree

First round `current_node =` **5**

```
_min_value_node(self, node):  
    current = node  
    while current.left is not None:  
        current = current.left  
    return current
```

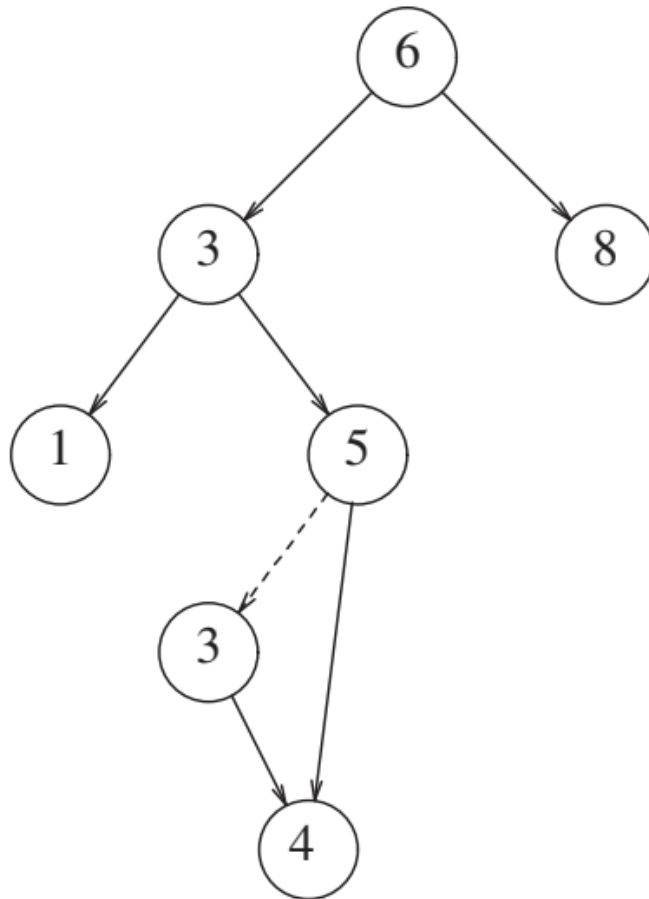


[Trace การทำงานของฟังก์ชัน `_min_value_node`]:

* การทำงานของ `_min_value_node(current_node.right)` เริ่มต้นที่ Node(5):

1. `current = node` (เริ่มต้นที่ Node(5))
 2. รอบที่ 1: ตรวจสอบ `while current.left is not None` (มี Node 3) -> True -> `current = current.left` (ไปยัง Node 3)
 3. รอบที่ 2: ตรวจสอบ `while current.left is not None` -> False (Node 3 ไม่มีลูกซ้าย)
 4. หลุดจาก Loop: คืนค่า `return current` (ส่งกลับ Node(3) ออกไป)
- > ผลลัพธ์: Node(3) เป็น In-order Successor (ค่าน้อยที่สุดใน Subtree ด้านขวา)





round 3: value = **3** , current_node = **5**

`current_node.left = self._delete_recursive(current_node.left, value)`

round 4: value = **3**, current_node = **3**

`elif current_node.left is None: return current_node.right`

Node(4) ถูกส่งกลับขึ้นไปแทนที่ตำแหน่งของ Node(3)

[สรุปผลลัพธ์และโครงสร้างต้นไม้หลังลบโหนด 2 สมบูรณ์]:

*** สรุปผลลัพธ์ขั้นตอนการลบโหนด 2 (มีลูก 2 ตัว) เสร็จสมบูรณ์:**

- ค่า 3 (In-order Successor) ถูกนำขึ้นมาแทนที่ตำแหน่งโหนด 2 เดิม
- ส่งกลับโหนด 3 ตัวเดิมออกจาก Subtree ด้านขวา (ลบแบบ One Child: โหนด 4 เลื่อนแทนที่)
- ตรวจสอบ In-order Traversal หลังลบ: [1, 3, 4, 5, 6, 8] ยังคงเรียงลำดับถูกต้องตามคุณสมบัติ BST ครบถ้วน

